Exercise 4: Employee Management System

1. Understand Array Representation

Arrays store elements in contiguous memory locations. Each element can be accessed directly using its index.

Advantages of Arrays

- Fast random access.
- Easy implementation.
- Memory efficient.
- Suitable when the number of records is known beforehand.

2. Setup

```
class Employee {
    int employeeId;
    String name;
    String position;
    int salary;

    Employee(int employeeId, String name, String position, int salary) {
        this.employeeId = employeeId;
        this.name = name;
        this.position = position;
        this.salary = salary;
    }
```

```
    public String toString() {
        return "Id: " + employeeId +
               " Name: " + name +
               " Position: " + position +
               " Salary: " + salary;
    }
}
```

3. Implementation

```
class Operations {

    public Employee[] add(Employee[] employees, Employee newEmployee) {
        Employee[] temp = new Employee[employees.length + 1];
```

```
        for (int i = 0; i < employees.length; i++) {
            temp[i] = employees[i];
        }
```

```
        temp[employees.length] = newEmployee;

        return temp;
    }

    public void search(Employee[] employees, int targetId) {
        for (int i = 0; i < employees.length; i++) {
            if (employees[i] != null &&
                employees[i].employeeId == targetId) {

                System.out.println("Employee " + employees[i] + " Found");
                return;
            }
        }

        System.out.println("Employee Not Found");
    }

    public void traverse(Employee[] employees) {
        for (Employee e : employees) {
            if (e != null) {
                System.out.println(e);
            }
        }
    }

    public void delete(Employee[] employees, int targetId) {
        for (int i = 0; i < employees.length; i++) {

            if (employees[i] != null &&
                employees[i].employeeId == targetId) {

                employees[i] = null;

                System.out.println("Employee Record Deleted");
                return;
            }
        }

        System.out.println("Employee Not Found");
    }
}

public class EmployeeManagementSystem {
    public static void main(String[] args) {

        Employee[] employees = {
            new Employee(101, "Alex", "Manager", 75000),
```

```
            new Employee(102, "John", "Developer", 60000),
            new Employee(103, "Emma", "Designer", 55000)
        };

        Operations op = new Operations();

        System.out.println("Initial Employees:");
        op.traverse(employees);

        employees = op.add(
            employees,
            new Employee(104, "Sophia", "Tester", 50000)
        );

        System.out.println("\nAfter Adding:");
        op.traverse(employees);

        System.out.println("\nSearching Employee 103:");
        op.search(employees, 103);

        System.out.println("\nDeleting Employee 102:");
        op.delete(employees, 102);

        System.out.println("\nAfter Deletion:");
        op.traverse(employees);
    }
}
```

4. Analysis

Operation	***Time Complexity***
Add Empbyee	O(n)
Search Employee	O(n)
Traverse Employees	O(n)
Delete Employee	O(n)

Limitations of Arrays

- Fixed size.
- Expensive insertion and deletion operations.
- Memory wastage if size is overestimated.
- Requires shifting elements after deletion.